

React

Les fondamentaux

C'est un composant ?

- Les composants sont un des concepts fondamentaux de **React**.
- Ils sont les fondations sur lesquelles sont construites les interfaces utilisateurs (UI).
- Un composant est une partie d'une UI
- Un composant est sensé être utilisé plusieurs à plusieurs endroits.
- Un composant peut être créé de deux façons différentes :

1. Les composants fonctions

```
export default function Helloworld(){  
  return(  
    <div>  
      <h1>Hello World</h1>  
    </div>  
  );  
}
```

2. Les composants class

- Les deux composants précédents sont équivalents du point de vue de React.
- Les noms des composants doivent commencer par une majuscule (donc les noms des fonctions)
- Noter la présence de la fonction **render()** dans le cas des classes
- Dans le composant racine, on ne peut lancer qu'un seul composant enfant.
- Les composants fonction sont plus utilisés que les fonctions class.

```
Bienvenue1.jsx >  Welcome
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

class Welcome extends React.Component {
  render() {
    return <h1>Bonjour, {this.props.name}</h1>;
  }
}
```

Composants et props

- Les composants permettent de découper l'interface utilisateur en éléments indépendants et réutilisables, afin de considérer chaque élément de manière isolée.
- Conceptuellement, les composants sont des fonctions JavaScript.
- Ils acceptent des entrées quelconques (appelées « props ») et renvoient des éléments React décrivant ce qui doit apparaître au niveau de l'UI.

Composant fonction

```
function Welcome(props) {  
  return <h1>Bonjour, {props.name}</h1>;  
}
```

Composant classe

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Bonjour, {this.props.name}</h1>;  
  }  
}
```

Passage des arguments aux composants

- Pour passer des paramètres à un composant, on utilise la notion les **props**.
- Conceptuellement, les composants sont comme des fonctions JavaScript. Ils acceptent des entrées quelconques (appelées "**props**") et renvoient des éléments **React** décrivant ce qui doit apparaître à l'écran.

```
//main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import ReactDOM from 'react-dom'
import './index.css'
import Bienvenue from './bienvenue.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <div>
      <Bienvenue promos="MMI3 - 2025" effectif="47" />
    </div>
  </StrictMode>,
)
```

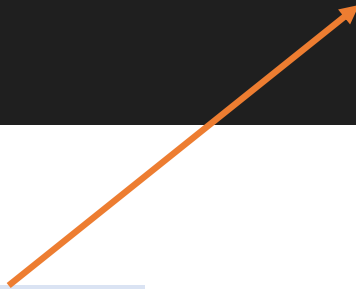
Passage des arguments aux composants

- Pour passer des paramètres à un composant, on utilise la notion les **props**.
- Conceptuellement, les composants sont comme des fonctions JavaScript. Ils acceptent des entrées quelconques (appelées "**props**") et renvoient des éléments **React** décrivant ce qui doit apparaître à l'écran.

```
//bienvenue.jsx
import ReactDOM from 'react-dom'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

export default function Bienvenue(props){
  return (
    <h1>Promotion : {props.promos} Effectif : {props.effectif}</h1>
  )
}
```

Argument 1



Argument 2



Imbrication des composants

- Les composants peuvent faire référence à d'autres composants dans leur sortie.
- On peut utiliser la même abstraction de composants pour n'importe quel niveau de détail. Un bouton, un formulaire, une boîte de dialogue, un écran : dans React, ils sont généralement tous exprimés par des composants.

```
//fichier App.jsx

import Bienvenue from "../bienvenue";

export function App() {
  return (
    <>
      <div>
        <Bienvenue promos="MMI1 - 2025" effectif="81" />
        <Bienvenue promos="MMI2 - 2025" effectif="62" />
        <Bienvenue promos="MMI3 - 2025" effectif="47" />
      </div>
    </>
  );
}

export default App
```

Mise à jour du fichier main.jsx

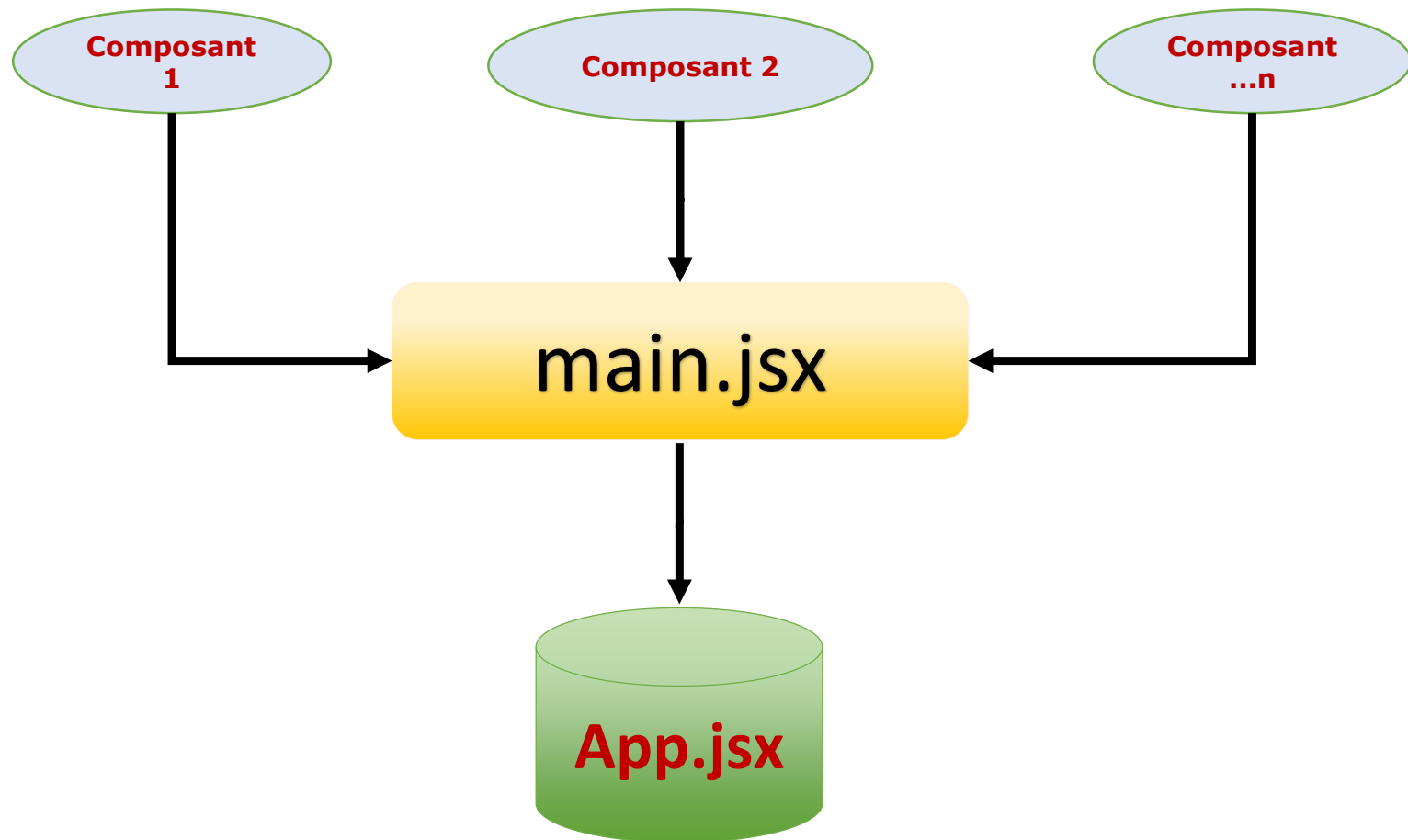
Le fichier **main.jsx** aura la forme suivante :

```
//main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import ReactDOM from 'react-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)
```


Utilisation des compoa

L'arborescence finale pour la création et l'utilisation des composants sous React est la suivante :



Utiliser des expressions JavaScript avec JSX

Dans JSX, les expressions JavaScript peuvent être intégrées directement dans le balisage pour générer du contenu de manière dynamique.

Cela permet aux développeurs d'utiliser du code JavaScript pour calculer des valeurs, effectuer des opérations et effectuer un rendu conditionnel du contenu dans leurs composants JSX.

```
import React from 'react';

const monCompo = () => {
  const nom = 'Ludovic';
  const age = 30;

  return (
    <div>
      <h1>Salut, {nom}!</h1>
      <p>Vous avez {age} ans.</p>
      <p>Votre âge l'an prochain sera de {age + 1} ans.</p>
      {age >= 18 && <p>VOus êtes majeur.</p>}
    </div>
  );
};

export default monCompo;
```

Utiliser des CSS avec JSX

Plusieurs manières permettent d'utiliser les CSS avec les composants JSX :

- Styles en ligne : Définis directement dans le code JSX
- Via un framework (Bootstrap, tailwind) → recommandé pour les grandes applications

```
const monCompo = () => {  
  const styles = {  
    backgroundColor: 'blue',  
    color: 'white',  
    padding: '10px'  
  };  
  
  return (  
    <div style={styles}>  
      <h1>Hello, World!</h1>  
      <p>Utilisation d'un style inline.</p>  
    </div>  
  );  
};  
  
export default monCompo;
```

Affichage conditionnel

L'affichage conditionnel en React fonctionne de la même façon que les conditions en Javascript.

On utilise l'instruction **Javascript if** ou l'opérateur ternaire pour créer des éléments représentant l'état courant, et on laisse React mettre à jour l'interface utilisateur (UI) pour qu'elle corresponde.

Exemple :

Soient les deux fonctions suivantes :

```
function Connexion(props) {  
  return <h1>Bienvenue !</h1>;  
}  
  
function Inscription(props) {  
  return <h1>Veuillez vous inscrire.</h1>;  
}
```

Affichage conditionnel

Il est possible de sélectionner l'une des deux fonctions, en lien avec un critère (test) :

```
function Compte(props) {  
  const estConnecte = props.estConnecte;  
  
  if (estConnecte) {  
    return <Connexion />;  
  }  
  
  return <Inscription />;  
}
```

Listes et les clés

Les listes sont très utilisées dans les cas d'affichage de plusieurs éléments dans un composant :

```
function NumberList(props) {  
  
  const numbers = props.numbers;  
  
  const listItems = numbers.map((number) =>  
    <li>{number}</li> );  
  return (  
    <ul>{listItems}</ul>  
  );  
}
```

Les événements

La gestion des événements sous React diffère légèrement de celle de Javascript, au niveau syntaxique :

```
import React from 'react'

export default function Button() {
  function handleClick() {
    alert('Vous avez cliqué!');
  }

  return (
    <button onClick={handleClick}>
      Cliquez ici
    </button>
  );
}
```

Passer une fonction (correct)

```
<button onClick={handleClick}>
```

Appeler une fonction (incorrect)

```
<button onClick={handleClick()}>
```

Les formulaires

Les formulaires sont réalisés sur React de la même façon qu'en HTML à quelques différences près.

```
const Formulaire = () => {  
  <tr>  
    <td className="fw-bold fs-6">Nom</td>  
    <td><input type="text" className="form-control mb-2" name='nom'  
      value={nom}  
      onChange={handleChangeNom} />  
    </td>  
  </tr>  
  <tr>  
    <td className="fw-bold fs-6">Prénom </td>  
    <td><input type="text" className="form-control mb-2" name='prenom'  
      value={prenom}  
      onChange={handleChangePrenom} />  
    </td>  
  </tr>  
  <tr>
```


Les formulaires

Il existe deux types de composants utilisés dans les formulaires :

- Composant **contrôlé** par React
- Composant **non contrôlé** par React, mais par le DOM

Il est recommandé d'utiliser les composants contrôlés.

```
const Controlled = () => {  
  return <input value="controlled" readOnly />;  
};  
  
const Uncontrolled = () => {  
  return <input defaultValue="Uncontrolled" />;  
};
```

Les formulaires

Dans un composant contrôlé, la modification ne peut se faire que par le biais d'un setter.

Pour un composé contrôlé, on donne la charge à React de venir modifier la valeur quand l'utilisateur change la valeur du champ. Il va se passer les étapes suivantes :

1. **onChange** est appelé
2. **setState** est appelé avec la nouvelle valeur
3. **React re-render** le composant avec la nouvelle valeur dans value

```
const Controlled = () => {  
  const [value, setValue] = useState("controlled");  
  return <input value={value} onChange={(e) => setValue(e.target.value)} />;  
};
```